

THE SURPRISING EFFECTIVENESS OF SHARED MEMORY IN LOOPED TRANSFORMERS

Giovanni Monea^{1*} Keshav Ramji² Yousef El-Kurdi² Luis A. Lastras²
Yoav Artzi¹ Nathan Godey^{1†} Ramón Fernández Astudillo^{2†}

¹Cornell University ²IBM Research

ABSTRACT

Looped Transformers apply the same layers several times per token, adding compute to improve quality without more parameters. Each recursion, however, writes its own key-value cache, so memory still grows with compute. Inference-time techniques can shrink this cache at a cost in quality. We pretrain looped language models to share memory: only the first recursion writes a cache, and later recursions read it while keeping a short window of their own. Surprisingly, we find that sharing memory does not cost quality and instead improves it. At 150M–1B parameters, our Looped Prediction Transformer (LPT) and its hybrid variant set a new quality–memory frontier for looped models: with five recursions, the hybrid lowers validation perplexity on FineWeb-Edu by 1.12–1.82 relative to a same-size standard Transformer while using 76–79% less context memory. Through an extensive analysis, we investigate why memory sharing helps. Shared and local memory develop different representations, and later recursions attend mostly to the shared memory, which also acts as a gradient highway to the first recursion.

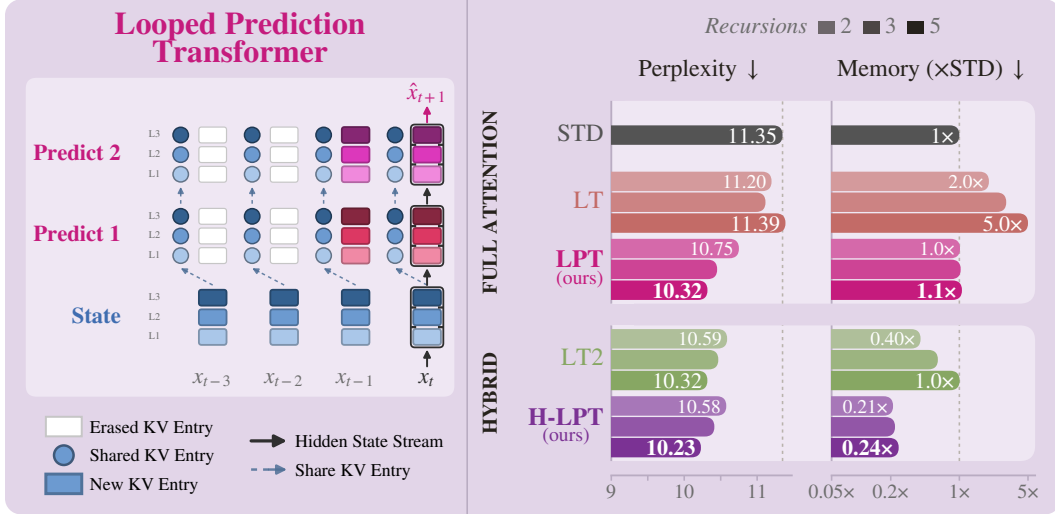


Figure 1: **Shared memory not only saves memory, it also improves quality.** **Left:** in our Looped Prediction Transformer (LPT), later recursions read the KV cache written by the first recursion and keep only a short window of their own. **Right:** perplexity and context memory of 1B-parameter models with two to five recursions, against a parameter-matched standard Transformer (STD). Each of our models adds shared memory to the looped baseline above it: LPT to the looped Transformer LT, and H-LPT to the gated DeltaNet hybrid LT2.

*Work done during an internship at IBM Research. Correspondence to: giovanni@cs.cornell.edu

†Equal advising

1 INTRODUCTION

Looped Transformers (LT) repeatedly apply the same layers several times before predicting the next token, adding computation without adding parameters (Dehghani et al., 2019; Zhu et al., 2025). In principle, they trade time for memory, matching the compute of a larger model with far fewer parameters. In standard formulations, however, context memory still grows with recursion depth, as each iteration keeps a separate key-value (KV) cache (Geiping et al., 2025; Zhu et al., 2025). At long contexts, this context memory can grow larger than the weights memory itself, undoing the very reason to use a looped model.

Sharing caches across recursions during decoding can save substantial memory with little quality loss (Geiping et al., 2025; Zhu et al., 2025). So far, this reuse has been studied mainly as a way to save memory. Yet it also shows that memory written by one recursion can serve the others, even in models never trained to share it. This raises a question: *if sharing can work untrained, what can it do when models are trained for it?* To answer it, we train looped models with memory sharing from the start, investigating it as an architectural inductive bias.

We study a simple recipe for *Memory Sharing* in looped Transformers (Figure 1, left). At each full-attention layer, later recursions attend to both their own KV entries and the cache written by that layer in the first recursion, within the same attention operation. To further save memory, we truncate the cache of every recursion after the first to a short sliding window. The resulting Looped Prediction Transformer (LPT) keeps LT’s sequential recurrence and uses context memory close to a standard (non-looped) Transformer’s. Applied to the full-attention layers of a hybrid looped Transformer (Deng et al., 2026), the same rule yields the Hybrid Looped Prediction Transformer (H-LPT). We pretrain all models from scratch at 150M to 1B parameters. Because later recursions give up most of their own memory, one would expect Memory Sharing to cost some quality, as untrained sharing does.

Surprisingly, Memory Sharing *improves* quality over full-cache LT even when each later recursion’s own cache covers less than 2% of the context. Its advantage grows with depth: LT stops improving in perplexity after three recursions, while LPT improves through five. Across sizes, LPT lowers validation perplexity by up to 1.09 nats and raises downstream accuracy by up to 4.8 percentage points over a matched-depth LT, with up to $4.7\times$ less context memory and nearly the same inference FLOPs. Sharing also improves the hybrid baseline: at five recursions, H-LPT outperforms a same-size standard Transformer in both language modeling and downstream accuracy while using 76–79% less context memory, moving the quality–memory frontier for looped models (Figure 2).

These results raise a further question: *why does memory sharing improve quality?* We find that later recursions rely heavily on the shared memory. LPT outperforms LT at every window size, even when later recursions keep no cache of their own, and later recursions attend mostly to shared memory even when their own cache covers the full context. Shared and local memory also develop distinct key representations, suggesting that they serve different purposes. Reading shared memory sends gradients directly to the first recursion, helping it learn what to store. This gradient highway bypasses the intermediate recursions: in LT, the magnitude of the gradient reaching the first recursion falls sharply with depth, while LPT recovers much of it through the shared memory. When we block this path, the model still reads shared memory but no longer matches LPT’s quality and stability.

2 LOOPED TRANSFORMERS

A standard LT shares its weights across recursions, while each recursion computes its own hidden states and KV entries. We use Ouro’s full-stack formulation (Zhu et al., 2025), which builds on earlier recurrent-depth architectures (Dehghani et al., 2019; Geiping et al., 2025).

Let $\mathcal{M}^L = \mathcal{T}_\theta^{(L)} \circ \dots \circ \mathcal{T}_\theta^{(1)}$ denote the shared stack of L Transformer blocks. For a sequence of length T and hidden dimension d , let $h_{\text{in}}^{(t)} \in \mathbb{R}^{T \times d}$ and $h^{(t)}$ be the residual streams entering and

leaving recursion t . The stack is applied R times:

$$\begin{aligned} h_{\text{in}}^{(1)} &= \text{emb}(\mathbf{x}), \\ h^{(t)} &= \mathcal{M}^L(h_{\text{in}}^{(t)}), \\ h_{\text{in}}^{(t+1)} &= \mathcal{N}_{\text{rec}}(h^{(t)}), \\ \text{logits}^{(t)} &= \text{lm_head}(\mathcal{N}_{\text{out}}(h^{(t)})), \quad t = 1, \dots, R. \end{aligned} \tag{1}$$

Here $\mathcal{N}_{\text{rec}}$ and $\mathcal{N}_{\text{out}}$ are separately parameterized RMSNorms. Both act on $h^{(t)}$: the recurrent norm prepares the next recursion’s input, while the readout norm feeds the language-modeling head. The effective depth is $L \cdot R$, with the parameters of a single L -layer stack.

Each layer computes its keys and values from the hidden states it receives at the current recursion. For a fixed layer, let $\mathcal{C}^{(t)}$ denote the cache written by recursion t , with entry $(k_j^{(t)}, v_j^{(t)})$ at position j . We suppress the layer index throughout: for vanilla LT, each layer reads only its own entries. A query at position i reads the causal prefix of its recursion’s cache, giving the attention set

$$\mathcal{A}_{\text{LT}}^{(t)}(i) = \mathcal{C}_{\leq i}^{(t)} = \{(k_j^{(t)}, v_j^{(t)}) : j \leq i\}. \tag{2}$$

At inference, each layer therefore retains R full-context caches, one per recursion, and context memory grows linearly with recursion depth even though the parameter count stays fixed. Some of these caches can be dropped or merged during decoding (Geiping et al., 2025; Zhu et al., 2025), but the model is still trained with a separate cache for every recursion.

Hybrid architectures reduce this memory cost by replacing most full-attention layers with linear attention, whose recurrent state does not grow with context length (Katharopoulos et al., 2020; Schlag et al., 2021; Yang et al., 2025). LT2 (Deng et al., 2026) loops such a stack, using Gated DeltaNet layers and one full-attention layer per five. This reduces the number of full-attention caches, but each remaining layer still needs a separate full-context cache at every recursion.

3 LOOPING WITH SHARED MEMORY

We study *Memory Sharing*, in which later recursions read the memory written by the first recursion, as an inductive bias for looped models. The first recursion runs as in LT and writes a full-context cache $\mathcal{C}^{(1)}$ at each layer, and every later recursion attends to this cache together with its own KV entries, using a single softmax.

To also keep looping memory-efficient, we restrict each later recursion’s own cache to a short window. The first recursion therefore acts as a memory phase that builds the model’s only full-context memory, while later recursions use it to refine the prediction. We refer to the first recursion as the *state recursion* and to the later ones as *prediction recursions*, and call the resulting model the Looped Prediction Transformer (LPT; Figure 1). Like LT, LPT applies the same L -layer stack R times and passes the final output to the language-modeling head (Equation 1).

Let ω be the number of preceding positions kept in each prediction recursion’s window, and write $\mathcal{C}_{[i-\omega, i]}^{(t)}$ for the entries from positions $i - \omega$ through i , including the current entry. The state recursion reads its own cache, as in LT (Equation 2). A prediction recursion reads the shared memory together with its own window. The entries read by a query at position i in recursion t are

$$\mathcal{A}_{\text{LPT}}^{(t)}(i) = \begin{cases} \mathcal{C}_{\leq i}^{(1)}, & t = 1, \\ \mathcal{C}_{\leq i}^{(1)} \cup \mathcal{C}_{[i-\omega, i]}^{(t)}, & t = 2, \dots, R. \end{cases} \tag{3}$$

In prediction recursions, the two sets are attended to using a single softmax.

Setting $\omega = T$ gives every prediction recursion a full-context cache of its own, as in LT, and leaves access to shared memory as the only difference between the two models; we use this setting in Section 6 to separate the effect of sharing from that of the short window.

Memory Sharing also applies to looped hybrid Transformers, which interleave linear-attention and full-attention layers (Yang et al., 2025; Deng et al., 2026). The first recursion runs the hybrid as usual. In later recursions, each linear-attention layer keeps its own recurrent state, which is small and fixed in size, while each full-attention layer applies Equation 3, as in LPT. Applied to LT2 (Deng et al., 2026), this gives H-LPT.

Table 1: **LPT and H-LPT reach lower perplexity than their looped baselines with much smaller caches.** Main results across S, M, and L: validation perplexity, mean five-task accuracy (%), and context memory and FLOPs relative to same-size STD. In the STD rows, $R = 1$ is the standard Transformer, and $R > 1$ is DEEPSTD. Elsewhere, R counts model passes. **Bold** and underline mark the best and second-best within each size and column group; our rows are shaded. We replicate the S results with three additional seeds and find the gains of Memory Sharing statistically significant ($p < 0.01$, Appendix B).

Size	Model	$R = 1$		$R = 2$				$R = 3$				$R = 5$			
		PPL ↓	Acc. ↑	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓
S	STD	16.83	51.5	17.07	50.2	1.50	1.19	17.27	50.7	1.88	1.31	17.69	50.4	2.50	1.54
	LOOPIE			16.58	51.6	2.00	1.83	16.50	51.9	3.00	2.66	16.56	52.1	5.00	4.32
	LT			16.17	51.5	2.00	1.83	15.98	52.1	3.00	2.66	16.00	52.4	5.00	4.32
	PLT			15.84	52.7	1.02	1.84	15.55	52.9	1.03	2.67	15.49	53.5	1.06	4.34
	LT2			<u>15.76</u>	51.9	<u>0.35</u>	<u>1.29</u>	15.47	52.5	<u>0.52</u>	<u>1.86</u>	15.16	53.7	<u>0.87</u>	2.98
	LPT			15.73	51.9	1.02	1.84	15.28	54.0	1.03	2.67	14.91	54.9	1.06	4.34
	H-LPT			15.73	<u>52.4</u>	0.18	<u>1.29</u>	<u>15.39</u>	<u>53.1</u>	0.19	<u>1.86</u>	<u>15.01</u>	<u>54.6</u>	0.21	<u>2.98</u>
M	STD	13.79	56.1	14.24	55.5	1.44	1.13	14.47	55.4	1.78	1.24	14.88	54.4	2.34	1.43
	LOOPIE			13.69	56.9	2.00	1.92	13.75	55.8	3.00	2.84	13.74	56.6	5.00	4.69
	LT			13.28	57.5	2.00	1.92	13.13	59.4	3.00	2.84	13.34	57.9	5.00	4.69
	PLT			13.00	<u>59.1</u>	1.02	1.93	12.68	60.0	1.03	2.85	12.65	60.5	1.06	4.71
	LT2			<u>12.81</u>	<u>59.1</u>	<u>0.35</u>	<u>1.43</u>	12.63	59.5	<u>0.52</u>	<u>2.10</u>	12.43	60.2	<u>0.87</u>	<u>3.45</u>
	LPT			12.91	59.2	1.02	1.93	<u>12.55</u>	<u>60.2</u>	1.03	2.85	<u>12.33</u>	61.6	1.06	4.70
	H-LPT			12.80	58.5	0.18	<u>1.43</u>	12.54	60.3	0.19	<u>2.10</u>	12.31	<u>61.4</u>	0.21	<u>3.45</u>
L	STD	11.35	62.5	11.62	62.5	1.45	1.14	11.96	61.3	1.73	1.17	12.27	61.4	2.25	1.32
	LOOPIE			11.26	63.5	2.00	1.95	11.28	63.9	3.00	2.91	12.26	61.0	5.00	4.81
	LT			11.20	64.2	2.00	1.95	11.11	63.8	3.00	2.91	11.39	63.4	5.00	4.81
	PLT			10.95	64.8	1.02	1.96	10.75	66.8	1.03	2.92	10.59	66.4	1.06	4.83
	LT2			<u>10.59</u>	66.5	<u>0.40</u>	<u>1.53</u>	10.47	65.8	<u>0.60</u>	<u>2.27</u>	<u>10.32</u>	67.3	<u>1.01</u>	<u>3.75</u>
	LPT			10.75	<u>66.3</u>	1.02	1.96	<u>10.45</u>	67.3	1.03	2.92	<u>10.32</u>	68.2	1.06	4.83
	H-LPT			10.58	65.6	0.21	<u>1.53</u>	10.41	<u>66.9</u>	0.22	<u>2.27</u>	10.23	<u>68.1</u>	0.24	<u>3.75</u>

Efficiency and Costs In training, we implement Equation 3 as a FlexAttention (Dong et al., 2025) block mask over the concatenated shared and local entries, which skips the fully masked blocks, so each query attends over the shared prefix and its ω window entries only (Appendix A).

During decoding, each prediction recursion keeps a sliding buffer of at most ω preceding entries per layer; its current entry participates in attention before entering the buffer. In cached token positions across L full-attention layers, the inference-time KV cache sizes are

$$M_{KV}^{LT} = LRT, \quad M_{KV}^{LPT} = L[T + (R - 1)\omega], \quad (4)$$

where the common head dimensions and datatype size are omitted. LPT uses the same parameters and recursion count as a matched-depth LT, but performs slightly more attention because each prediction recursion also reads its local window. At a full context of $T = 4,096$ with $\omega = 64$ and five recursions, these windows increase the number of attended KV entries per generated token by only $(R - 1)\omega/(RT) = 1.25\%$, while reducing the full-attention cache relative to LT by $4.7\times$.

4 EXPERIMENTAL SETUP

Baselines To study memory sharing, we compare LPT with LT and H-LPT with LT2. We also compare with the Parallel Loop Transformer (PLT) (Wu et al., 2025), which parallelizes recursions and uses gated dual attention over the first recursion’s shared cache and each later recursion’s local cache. Other baselines are a standard Transformer (STD), LOOPIE, which repeats each layer before advancing to the next (Gao et al., 2026), and DEEPSTD, a deeper, narrower Transformer without weight sharing that matches the looped models in effective depth and approximately in parameters.

Architecture All models use a decoder-only Transformer backbone with RMSNorm, SwiGLU, rotary positional embeddings, and tied input and output embeddings. We train S, M, and L models

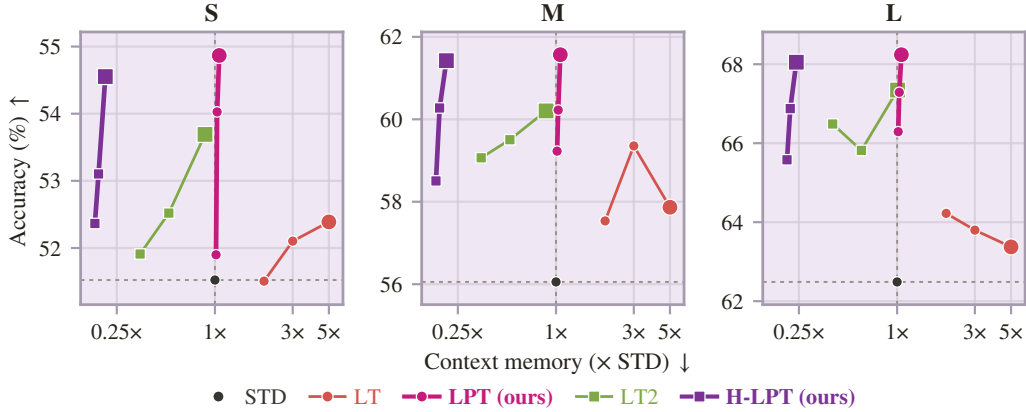


Figure 2: **LPT and H-LPT reach higher accuracy with much smaller caches than the looped baselines.** Mean five-task accuracy vs. context memory at each size. Curves follow two, three, and five recursions, with larger markers at five. Dashed lines mark single-pass STD.

with approximately 150M, 450M, and 1B parameters; the main comparisons cover all three sizes, and most ablations and analyses use S and M. The context length is 4,096. Unless otherwise specified, LPT and H-LPT use a prediction window of $\omega = 64$. We study two, three, and five recursions. This range includes the two loop steps of LOOPIE (Gao et al., 2026) and surrounds the four recursions of Ouro and LT2 (Zhu et al., 2025; Deng et al., 2026).¹ All models use nanoGPT’s default optimizer settings (Karpathy, 2022). Model dimensions, normalization, and training details are in Appendix A.

Data and Training We pretrain on FineWeb-Edu (Penedo et al., 2024) using 20B tokens for S and M, and 50B for L, roughly 2–7× the Chinchilla compute-optimal token budgets (Hoffmann et al., 2022). The main tables report one run per configuration, with the same initialization seed and data order across models. At S, we also train three more seeds per model and confirm that seed variation is far below the gaps we report (Appendix B).

We study two training objectives. *Final loss* computes the next-token NLL at the final recursion only. *Adaptive loss* follows Ouro (Zhu et al., 2025), weighting per-recursion NLLs with a learned per-token halting distribution and adding KL regularization. Unless otherwise specified, LT uses adaptive loss and all other models use final loss.

Evaluation We report perplexity on 10M held-out FineWeb-Edu tokens and mean zero-shot accuracy on ARC-Easy (Clark et al., 2018), HellaSwag (Zellers et al., 2019), PIQA (Bisk et al., 2020), SciQ (Welbl et al., 2017), and LAMBADA (Paperno et al., 2016), using the LM Evaluation Harness (Gao et al., 2023). We compute context memory and decoding FLOPs per generated token analytically at a 4,096-token context, relative to the same size STD. Context memory includes the KV cache and GDN’s recurrent state.

5 RESULTS

Memory Sharing Improves Quality Restricting the memory written by later recursions could hurt prediction. Surprisingly, our Memory Sharing approach improves quality. At matched size and recursion count, LPT lowers validation perplexity by 0.38–1.09 and raises accuracy by 0.4–4.8 percentage points over LT, with 2.0–4.7× less KV memory and under 1% more FLOPs (Table 1). Measured on an H100, the memory savings are the same, so LPT fits 2.0–4.8× as many sequences in a batch as LT (Appendix F). The perplexity gain grows with recursion count (Table 1): LT stops improving beyond three recursions at every size, while LPT continues to improve through five. At comparable cache sizes, LPT also reaches lower perplexity than PLT’s parallel formulation.

¹Nine recursions bring instability in several model families and limited accuracy gains, consistent with the loss spikes Ouro reports at eight (Zhu et al., 2025). We report these experiments in Appendix E.

When it comes to hybrid variants, H-LPT keeps this memory at $0.18\text{--}0.24\times$ STD’s across recursion counts, while LT2’s remaining full-attention caches still grow with depth, approaching STD’s context memory at five recursions. Despite these $2\text{--}4\times$ memory gains, H-LPT matches or outperforms LT2 in terms of quality at every recursion count. At five recursions, it lowers perplexity by $0.09\text{--}0.15$ and raises accuracy by $0.8\text{--}1.2$ points over LT2, with $4.1\text{--}4.2\times$ less context memory and about 0.1% more inference FLOPs.

Incidentally, we note that for low recursion counts, the hybrid LT2 improves on LT while using less memory and compute. A factor that could explain this gap – and the instability of LT – is Ouro’s normalization, which we use for both LT and LPT. In Appendix C, we explore other normalizations that make LT more robust, but none of these variants matches LPT.

More Quality from the Same Parameters Both memory-sharing models improve on STD with little additional memory, or much less for H-LPT (Figure 2). At five recursions, LPT lowers perplexity by $1.03\text{--}1.92$ and raises accuracy by $3.4\text{--}5.7$ points with 6.25% more context memory. H-LPT lowers perplexity by $1.12\text{--}1.82$ and raises accuracy by $3.1\text{--}5.6$ points with $76\text{--}79\%$ less memory. This smaller cache lets H-LPT fit $3.6\text{--}5.0\times$ as many sequences as STD on one GPU. With three to five recursions, H-LPT lowers perplexity by $0.94\text{--}1.82$ while beating or staying close to STD’s throughput ($1.4\text{--}1.5\times$ at three, $81\text{--}88\%$ at five, Appendix F). Both models’ accuracy gains grow from two to five recursions at every size. LPT also outperforms LOOPIE and DEEPSTD, neither of which improves consistently with depth.

Takeaway 1. Through Memory Sharing, LPT and especially H-LPT improves the quality–memory frontier for looped models (Figure 2).

6 WHY MEMORY SHARING HELPS

We use S- and M-size models with two, three, and five recursions to study why this inductive bias improves quality (Appendix D).

Memory Sharing Stabilizes Deep Recursion We compare final and adaptive loss in LT and LPT (Figure 3). Without intermediate supervision, adding recursions consistently worsens LT’s perplexity, and the model collapses to unigram-level loss at five recursions. LPT stays stable, and final loss lowers its perplexity by $0.16\text{--}0.50$ compared with adaptive loss. Its intermediate recursions can refine the hidden state without having to predict early.

Takeaway 2. LPT stays stable and trains best with final loss.

Prediction Recursion Needs Local Memory We train LPT with prediction windows ranging from $\omega = 0$ to the full context of 4,096 tokens (Figure 4). LPT outperforms LT at every window size, across both sizes and all tested depths. This includes $\omega = 0$, where prediction recursions keep no local history and read shared memory alongside only their current KV entry. At the other extreme, $\omega = 4,096$, LT and LPT differ only in access to shared memory. LPT still lowers perplexity by $0.24\text{--}1.15$ relative to LT with the better of its two training objectives at each depth, close to the $0.32\text{--}1.09$ gain at $\omega = 64$. Overall, window size has a modest effect on quality, and what really matters is the access to the shared memory. To balance quality and efficiency, we adopt $\omega = 64$ as the default for our memory sharing models.

Takeaway 3. Shared memory improves quality across prediction-window sizes, even when later recursions keep no local history.

Later Recursions Attend More to Shared Memory We next measure how much attention is assigned to the shared memory in later recursions. We evaluate each model on the same 4,096-token validation sequence and average the attention assigned to shared keys over queries and heads (Figure 5

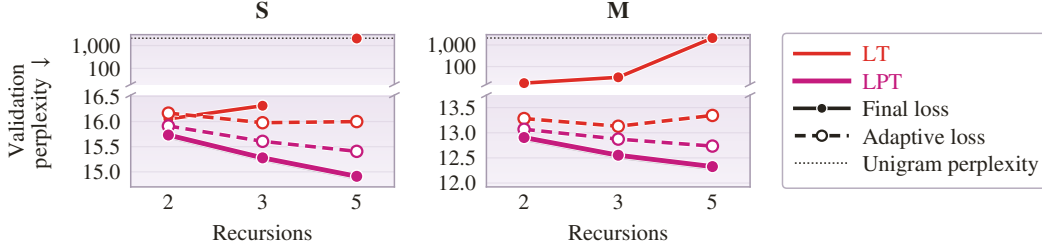


Figure 3: **LPT trains best with final loss, while LT consistently worsens with more recursions without adaptive loss.** Validation perplexity ($\downarrow$), collapsed runs in the upper strip on a log scale. Filled markers are final loss, hollow ones adaptive loss.

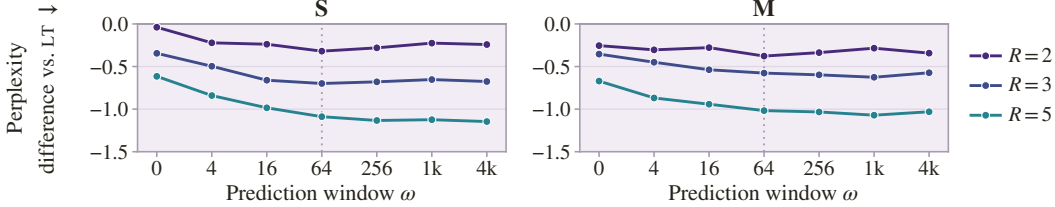


Figure 4: **A short prediction window is enough.** Validation perplexity of LPT relative to LT ($\downarrow$) vs. window size, LT at its better objective per depth. The dotted line marks $\omega = 64$.

and Appendix D). Shared memory receives most attention at every prediction recursion in both sizes. Its share is 57–71% in earlier prediction recursions and rises to 73–80% in the final recursion.²

Takeaway 4. Later recursions attend more to shared memory than to their own caches, with a stronger preference when getting closer to the prediction.

Shared and Local Memory Develop Different Representations Shared keys serve every later recursion, whereas local keys serve only the recursion that writes them. To see whether these roles are reflected in their representations, we compare S-size LT and LPT with three recursions, both trained with final loss, the only tested setting with at least three recursions where LT trains with final loss without a substantial drop in quality (Figure 3). We evaluate both models on the same 4,096-token validation sequence and extract queries and keys before rotary encoding. We concatenate the attention heads and fit principal component analysis (PCA) jointly to queries and keys across tokens and recursions per model and layer (Appendix D).

Shared and local keys consistently form distinct clusters in LPT, whereas keys from different recursions overlap more in LT (Figure 6). In Appendix D, we show similar separation in LPT across S and M at three and five recursions (Figure 10). We also show that with full prediction caches, the clusters overlap more but still remain distinguishable, even though shared and local keys cover the same positions (Figure 9). Prior work attributes the small cost of cache reuse to the similarity of keys and values across recursions (Bae et al., 2025), as we see in LT. With Memory Sharing, shared and local keys are instead clearly different. This shows that representation similarity across layers is not a necessary condition for effective memory sharing. This also fits the freedom that final loss gives: recursions need not all produce prediction-ready states, so the first can specialize in building memory and the later ones in using it. This specialization may help explain the quality gains, similar to how Monea et al. (2026) show that Transformers benefit from assigning memory construction and next-token prediction to separate residual streams.

Takeaway 5. Shared and local memory diverge in their local representations, so sharing is not reliant on recursions writing similar memories.

²Additional comparisons in the appendix show that shared memory still receives most attention with full prediction caches, while adaptive loss makes its use more uniform across recursions (Figure 11).

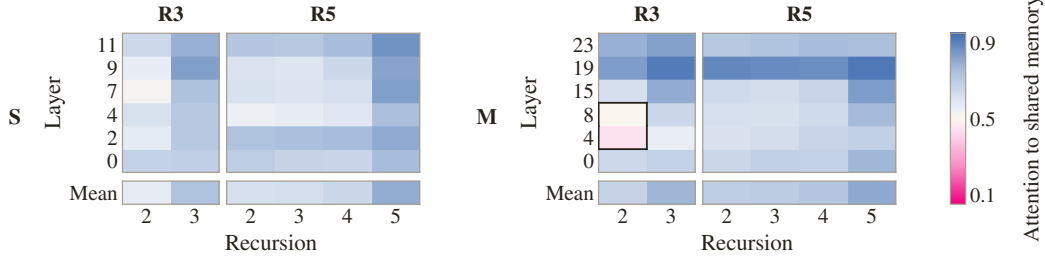


Figure 5: **Prediction recursions attend mostly to shared memory.** Attention mass assigned to shared KV entries in LPT, averaged over queries and heads. For clarity, only six layers are displayed. The bottom row averages all layers.

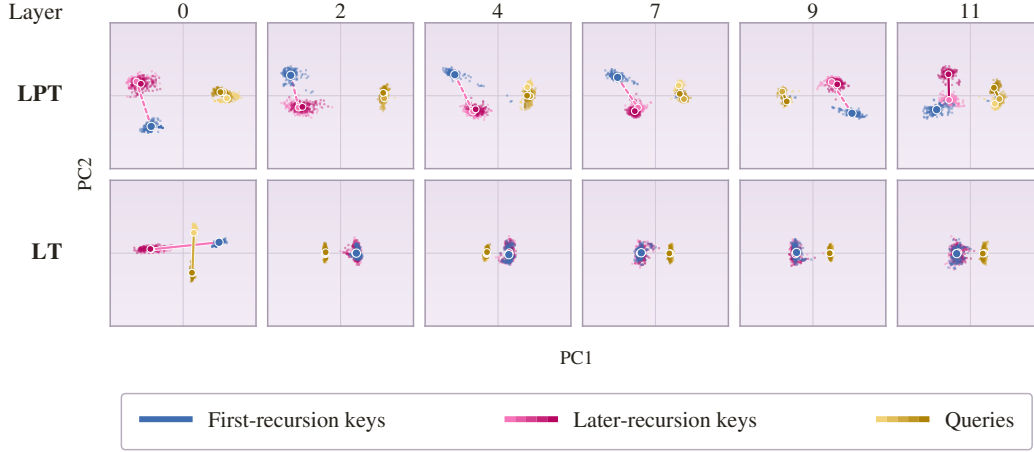


Figure 6: **Shared and local keys separate in LPT.** Joint query/key PCA, fitted separately for each panel. Dots are tokens and large circles their means; darker shades mark later recursions.

Shared Memory Is a Gradient Highway Reading shared memory gives the final loss a direct gradient path to the first recursion, bypassing the intermediate recursions. We evaluate the STD, LT, and LPT models of Table 1 on the same 32 validation sequences of 4,096 tokens and backpropagate the final loss from 32 fixed positions in each. We measure each recursion’s contribution to the gradient on the Transformer block weights, and split the first recursion’s contribution into its hidden-state and shared-KV paths by repeating the backward pass with shared KV detached (Appendix D).

In LT, the first-recursion gradient norm falls sharply with depth. LPT recovers much of the signal magnitude relative to the single-pass STD reference, with a much smaller decline as depth increases (Figure 7). At five recursions, most of this gradient comes through the shared KV cache. The same holds for LPT with adaptive loss or full prediction caches (Figure 12).

To test whether this highway helps learning, we train LPT from scratch with shared KV detached at later reads. The forward computation and hidden-state gradient path are unchanged. Perplexity rises in both sizes and at every depth, and deeper detached models often fail to train (Figure 8).

Takeaway 6. The memory sharing mechanism creates a gradient highway from the final loss to the first recursion that facilitates learning.

7 RELATED WORK

Separating Memory from Prediction In a standard Transformer, each hidden state does two jobs. It predicts the next token, and it writes the keys and values that later positions read. Several architectures separate these jobs across token streams, as in the two-stream attention of XLNet (Yang et al., 2019). The State-Prediction Separation Transformer (Monea et al., 2026) gives memory

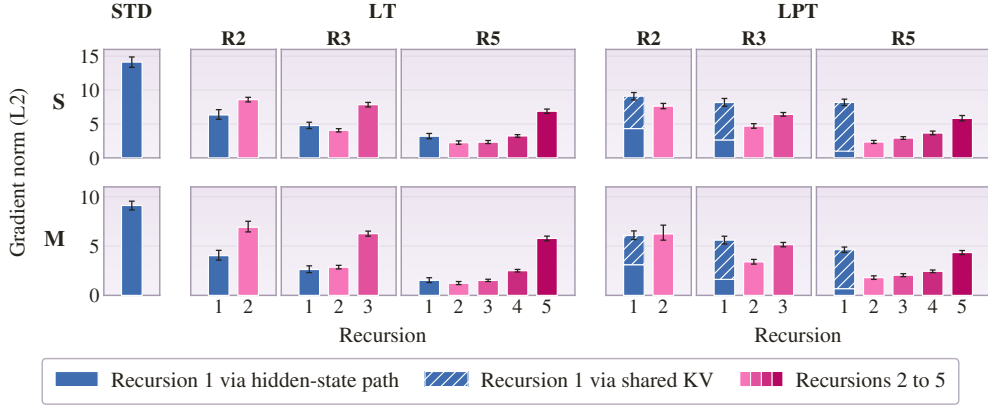


Figure 7: **Shared memory restores much of the gradient reaching the first recursion.** Mean gradient norm contributed by each recursion, with single-pass STD as reference. The first LPT bar is split into shared-KV (hatched) and hidden-state projections. LT is trained with adaptive loss, the others with final loss. Whiskers are 95% sequence-bootstrap intervals.

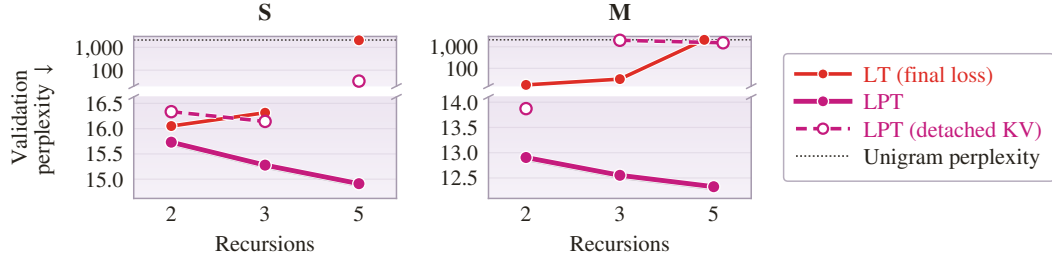


Figure 8: **Detaching shared-cache gradients hurts learning.** Validation perplexity (↓), collapsed runs in the upper strip on a log scale. All models are trained with final loss; Table 6 lists the values.

construction and next-token prediction separate residual streams and improves language modeling, and Langford et al. (2026) study a variant of this split. A second family separates them across layers. The first layers write one cache once, and the remaining layers only read it (Sun et al., 2024; 2026; DeepSeek-AI, 2026; Capps, 2026). Instead, LPT decouples memory from prediction across the recursions of a looped Transformer: the first recursion writes the model’s only full-context memory, and every later recursion reads it while it predicts, keeping just a short window of its own. Looped models thus offer a third place for the split, with no extra stream or layer, and our analysis finds evidence that the recursions learn these roles (Section 6).

Looped Transformers and Memory Sharing Looped Transformers apply the same layers several times (Dehghani et al., 2019; Geiping et al., 2025; Zhu et al., 2025), and recent work differs in which layers repeat and how recursions connect (Gao et al., 2026; Wang et al., 2026a; Huang et al., 2026; Mozer et al., 2026; Wang et al., 2026b; Deng et al., 2026). In standard formulations, each recursion writes its own KV cache, so context memory grows with depth (Geiping et al., 2025; Zhu et al., 2025). Sharing a cache across recursions, like sharing it across layers (Brandon et al., 2024), has been studied as a way to save memory. Without retraining, Huginn (Geiping et al., 2025) reuses caches cyclically with little quality loss, and Ouro (Zhu et al., 2025) can keep only the final or an averaged cache. Keeping only the first recursion’s cache sharply reduces Ouro’s quality, which suggests that shared memory has to be trained. Trained variants reuse the first recursion’s keys and values at a slight cost in quality (Bae et al., 2025), compute them once in a separate prelude (Capps, 2026), or learn gated or compressed caches (Vendrell et al., 2026; O’Neill & Reid, 2026). PLT (Wu et al., 2025) has the closest design to ours, although it focuses on parallelizing recursions. It shares the first recursion’s cache with gated local caches and runs recursions in parallel, while LPT keeps them sequential and reads both caches in one softmax. We find that LPT performs better. All of these treat sharing as a cost to contain. We instead treat it as an inductive bias that separates memory from prediction and show that it improves quality (Section 5 and Section 6).

8 DISCUSSION

We investigate memory sharing as an architectural inductive bias for looped models, and find that it not only cuts context memory but also improves quality in both full attention and hybrid models. Our Memory Sharing LPT models achieve consistently better perplexity and downstream accuracy than their usual looped counterparts, while matching the KV cache memory of a standard non-looped model. Our analyses suggest that sharing helps recursions learn different roles. The first recursion builds memory that later recursions use to refine the prediction. Those reads also give the final loss a direct gradient path to the first recursion. The different representations of shared and local memory support this division of roles, and cutting this gradient path hurts learning. Shared memory changes how later recursions read information and how the first recursion learns to write it.

Overall, our findings suggest that shared memory is a useful inductive bias for looped models, helping them make better use of computation within a limited memory budget.

Limitations & Future work Like other looped Transformers, LPT is not compute-optimal: it trades time for quality, but unlike LT, its memory barely grows with depth. At five recursions it costs $4.3\text{--}4.8\times$ the decoding FLOPs of STD, about the same as LT, and it spends the same number of recursions on every token. Choosing that number per token is future work. All our comparisons are parameter-matched, including DEEPSTD, and we do not train a standard Transformer matched in decoding FLOPs instead, which at five recursions would have several times the parameters. We train up to 1B parameters on one corpus with 4,096-token contexts and do not test long-context tasks or, given our limited pretraining, very advanced tasks. Our analyses use S and M models, since not every configuration they need was trained at L, and only the detach ablation looks at training dynamics. Both the stability and memory improvements of LPT are promising for effectively scaling looped models to higher recursion counts. More sophisticated Memory Sharing approaches could also be explored in future work, such as increasing the number of state recursions or alternating state recursions and prediction recursions.

ACKNOWLEDGMENTS

We thank Aviv Ron, Brian Kingsbury, and David Cox for useful feedback on internal presentations of this work.

REFERENCES

- Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/8b08bbf8b420faa6eeb4020720582ec7-Abstract-Conference.html.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6239>.
- William Brandon, Mayank Mishra, Aniruddha Nrusingha, Rameswar Panda, and Jonathan Ragan-Kelley. Reducing transformer key-value cache size with cross-layer attention. In *Advances in Neural Information Processing Systems*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/9e23d020c18e4c40d81c6a0fc7a46f68-Abstract-Conference.html.
- Chad A. Capps. CART: Context-anchored recurrent transformer – a parameter-efficient architecture with learned stability, 2026. URL <https://arxiv.org/abs/2606.01495>.
- Chen Chen and Lai Wei. Post-LayerNorm is back: Stable, ExpressivE, and deep, 2026. URL <https://arxiv.org/abs/2601.19895>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the AI2 Reasoning Challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
- DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression, 2026. URL <https://arxiv.org/abs/2609.19969>.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal Transformers. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HyzdRiR9Y7>.
- Chunyuan Deng, Yizhe Zhang, Rui-Jie Zhu, Yuanyuan Xu, Jiarui Liu, T. S. Eugene Ng, and Hanjie Chen. LT2: Linear-time looped transformers, 2026. URL <https://arxiv.org/abs/2605.20670>.
- Ming Ding, Zhuoyi Yang, Wenyi Hong, Wendi Zheng, Chang Zhou, Da Yin, Junyang Lin, Xu Zou, Zhou Shao, Hongxia Yang, and Jie Tang. CogView: Mastering text-to-image generation via transformers. In *Advances in Neural Information Processing Systems*, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/a4d92e2cd541fca87e4620aba658316d-Abstract.html.
- Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. FlexAttention: A programming model for generating fused attention variants. In *Proceedings of Machine Learning and Systems*, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/61a9278dfef5f871b5e472389f8d6fa1-Abstract-Conference.html.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation. Zenodo, version v0.4.0, December 2023. URL <https://zenodo.org/records/10256836>.
- Zitian Gao, Yilong Chen, Yihao Xiao, Xinyu Yang, Ran Tao, Joey Zhou, and Bryan Dai. Loop the Loopies!, 2026. URL <https://arxiv.org/abs/2607.16051>.

- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3b01972cf31e6fa0fe29e4b8b5c2a0a1-Abstract-Conference.html.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pp. 30016–30030, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/c1e2faff6f588870935f114ebe04a3e5-Abstract-Conference.html.
- Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. Towards looped models done right—Part I: Topology, input injection, recurrent-state design. Institute of Foundation Models technical blog, 2026. URL <https://ifm-research.notion.site/3ade511912ec8128987dfef7a5580043>. Accessed September 16, 2026.
- Andrej Karpathy. nanoGPT. GitHub repository, commit 325be85, 2022. URL <https://github.com/karpathy/nanoGPT>.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 5156–5165, 2020. URL <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- John Langford, Nathan Godey, Giovanni Monea, Yoav Artzi, Harry Dong, Ying Fan, Gustavo de Rosa, and Zheng Zhan. Almost free state prediction separation, 2026. URL <https://arxiv.org/abs/2609.03807>.
- Giovanni Monea, Nathan Godey, Kianté Brantley, and Yoav Artzi. The state-prediction separation hypothesis. In *Advances in Neural Information Processing Systems*, 2026. URL <https://arxiv.org/abs/2607.01218>.
- Michael C. Mozer, Shoaib Ahmed Siddiqui, Danny Sawyer, Sunny Sanyal, and Rosanne Liu. Recirculation, 2026. URL <https://arxiv.org/abs/2608.17981>.
- James O’Neill and Fergal Reid. Looped latent attention: Cross-loop KV compression for looped transformers, 2026. URL <https://arxiv.org/abs/2607.15456>.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1525–1534, 2016. URL <https://aclanthology.org/P16-1144/>.
- Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 37, pp. 30811–30849, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/370df50ccfd8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 9355–9366, 2021. URL <https://proceedings.mlr.press/v139/schlag21a.html>.

- Noam Shazeer. GLU variants improve Transformer, 2020. URL <https://arxiv.org/abs/2002.05202>.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063. URL <https://doi.org/10.1016/j.neucom.2023.127063>.
- Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models. In *Advances in Neural Information Processing Systems*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/0df38cd13520747e1e64e5b123a78ef8-Abstract-Conference.html.
- Yutao Sun, Li Dong, Tianzhu Ye, Shaohan Huang, Jianyong Wang, and Furu Wei. Universal YOCO for efficient depth scaling, 2026. URL <https://arxiv.org/abs/2604.01220>.
- Victor Conchello Vendrell, Arnau Padrés Masdemont, Niccolò Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. Memory-efficient looped transformer: Decoupling compute from memory in looped language models, 2026. URL <https://arxiv.org/abs/2605.07721>.
- Shaowen Wang, Ge Zhang, Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, and Jian Li. SMELT: Scaling laws for compute-matched MoE looped transformers, 2026a. URL <https://arxiv.org/abs/2609.01343>.
- Xi Wang, Ziyang Cai, Zheng Zhan, Harry Dong, Ying Fan, Gustavo de Rosa, Tim Pearce, and John Langford. Full-bandwidth transformer, 2026b. URL <https://arxiv.org/abs/2608.08888>.
- Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text*, pp. 94–106, 2017. URL <https://aclanthology.org/W17-4413/>.
- Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel loop transformer for efficient test-time computation scaling, 2025. URL <https://arxiv.org/abs/2510.24824>.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 10524–10533, 2020. URL <https://proceedings.mlr.press/v119/xiong20b.html>.
- Songlin Yang and Yu Zhang. FLA: A Triton-based library for hardware-efficient implementations of linear attention mechanism. GitHub repository, 2024. URL <https://github.com/fla-org/flash-linear-attention>.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, pp. 29687–29707, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>.
- Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. XLNet: Generalized autoregressive pretraining for language understanding. In *Advances in Neural Information Processing Systems*, 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/dc6a7e655d7e5840e66733e9ee67cc69-Abstract.html.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800, 2019. URL <https://aclanthology.org/P19-1472/>.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling latent reasoning via looped language models, 2025. URL <https://arxiv.org/abs/2510.25741>.

A ARCHITECTURE AND TRAINING DETAILS

Architecture Table 2 lists the base model dimensions. The decoder-only backbone uses RMSNorm pre-normalization (Zhang & Sennrich, 2019), SwiGLU feed-forward networks (Shazeer, 2020), no biases in linear layers, and tied input and output embeddings. Full-attention layers use rotary positional embeddings (Su et al., 2024).

Table 2: **Base model configurations.** Looped models reuse the listed L -layer stack.

	S	M	L
Layers (L)	12	24	36
Hidden size (d)	768	1,024	1,280
Heads	12	16	20
FFN size	3,072	4,096	5,120
Parameters	151.9M	454.2M	1.01B

LT and LPT use Ouro’s normalization and recurrent transition (Zhu et al., 2025). LT2 and H-LPT use the Full+GDN layer pattern and gating setup of Deng et al. (2026), with one full-attention layer per five layers. The full-attention layers read the complete causal context in LT2. In H-LPT, later recursions read the first recursion’s full-context cache and their own local prediction window, as in Equation 3.

PLT uses the same block, recurrent, and readout normalization as LT and LPT. Its recurrent transition adds the input embedding to the previous recursion’s normalized output shifted by one token.

Attention Implementation During training, each prediction recursion concatenates its keys and values with the shared keys and values along the sequence dimension, separately at each full-attention layer. A block mask implements Equation 3, allowing full causal attention over shared entries and causal sliding-window attention over local entries. FlexAttention (Dong et al., 2025) skips fully masked blocks.

Data Processing We tokenize with the GPT-2 tokenizer (Radford et al., 2019) and pack documents into 4,096-token sequences, inserting an end-of-sequence token between documents. Attention is masked across document boundaries, and boundary tokens are excluded from the language-modeling loss.

Optimization We use AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, and gradient clipping at 1.0. The global batch contains 96 sequences of 4,096 tokens, and training uses bfloat16 mixed precision. For L-size models, AdamW optimizer state is sharded across DDP ranks (ZeRO-1). The learning rate increases linearly to 6×10^{-4} over the first 50M tokens, stays constant, then decreases linearly to 6×10^{-5} over the final 10% of training. These are nanoGPT’s default optimizer settings, used unchanged for every model, size, and objective: we did not tune the learning rate or the warmup for any model, and we did not test whether a lower peak learning rate or a longer warmup changes how final-loss LT trains at depth (Figure 3).

For adaptive loss, we use Ouro’s adaptive-exit objective (Zhu et al., 2025). A learned per-token halting distribution weights the losses across recursions, with a KL regularization weight of 0.1. Final loss applies the language-modeling loss only at the last recursion.

B SEED VARIANCE AND STATISTICAL TESTS

Retraining every configuration at every size with several seeds is prohibitive at pretraining cost, so we run the check at S. Each S-size model of Table 1 is trained with four seeds: the run reported in the table and three more, each changing the initialization and the data order together. All other settings stay at their main-table values. Table 3 lists the mean and 95% confidence interval (Student- t) of the final validation perplexity for every model and recursion count. Table 4 tests each gap of interest with a one-sided paired t -test on the same-seed differences, with the alternative that the first model has lower perplexity. The largest standard deviation of perplexity across seeds is 0.08, against gaps of 0.40–1.05 between LPT and LT, and every difference keeps its sign at every seed.

Table 3: **Seed variance at S.** Final validation perplexity, mean $\pm$ 95% confidence interval (Student- t) over four seeds. In the STD row, $R = 1$ is the standard Transformer and $R > 1$ the deeper, narrower parameter-matched Transformer. SPS is the two-recursion baseline of Appendix E.

Model	$R = 1$	$R = 2$	$R = 3$	$R = 5$
STD	16.84 ± 0.03	17.03 ± 0.05	17.27 ± 0.07	17.64 ± 0.13
SPS		16.12 ± 0.07		
LOOPIE		16.57 ± 0.03	16.57 ± 0.08	16.58 ± 0.03
LT		16.16 ± 0.03	16.00 ± 0.05	15.99 ± 0.03
PLT		15.83 ± 0.05	15.55 ± 0.03	15.46 ± 0.03
LT2		15.77 ± 0.02	15.47 ± 0.02	15.16 ± 0.01
LPT		15.76 ± 0.07	15.29 ± 0.04	14.94 ± 0.04
H-LPT		15.73 ± 0.02	15.40 ± 0.02	15.03 ± 0.02

Table 4: **Paired tests at S.** Same-seed perplexity gap of the first model over the second, mean $\pm$ 95% confidence interval over four seeds, the gap in units of the standard deviation of the same-seed differences, and the one-sided paired t -test for the first model having lower perplexity.

Comparison	R	Gap (PPL)	Gap/SD	p
LPT vs. LT	2	-0.40 ± 0.05	13	6.5×10^{-5}
	3	-0.71 ± 0.05	24	9.4×10^{-6}
	5	-1.05 ± 0.04	39	2.4×10^{-6}
H-LPT vs. LT2	2	-0.04 ± 0.01	4	1.7×10^{-3}
	3	-0.07 ± 0.02	5	9.8×10^{-4}
	5	-0.13 ± 0.02	9	1.6×10^{-4}
LPT vs. PLT	2	-0.07 ± 0.09	1	4.8×10^{-2}
	3	-0.27 ± 0.07	6	5.4×10^{-4}
	5	-0.52 ± 0.07	11	9.7×10^{-5}

C NORMALIZATION BETWEEN RECURSIONS

Our LT and LPT use the Ouro paper’s normalization (Zhu et al., 2025). Sandwich norms normalize the output of every sublayer before the residual addition, and the recurrent norm $\mathcal{N}_{\text{rec}}$ normalizes each recursion’s output before the next recursion (Equation 1). With final loss, this LT degrades or diverges at depth (Figure 3), so the main tables train it with adaptive loss. We did not tune the normalization of either model. To test whether Ouro’s normalization holds LT back, we also train S and M LT with final loss and five other normalizations, each shared across recursions:

- *Pre-norm only* (Xiong et al., 2020) removes both, so $h_{\text{in}}^{(t+1)} = h^{(t)}$ and each block keeps only its pre-norms.
- *Sandwich only* keeps the sandwich norms (Ding et al., 2021) and removes $\mathcal{N}_{\text{rec}}$.
- *Input norm* removes the sandwich norms and keeps $\mathcal{N}_{\text{rec}}$, which it also applies to the embeddings.
- *Update norm* removes the sandwich norms and normalizes only each recursion’s update, so the carried state stays unnormalized: $h_{\text{in}}^{(t+1)} = h_{\text{in}}^{(t)} + \mathcal{N}(h^{(t)} - h_{\text{in}}^{(t)})$.
- *KEEL* removes the sandwich norms and applies the highway residual of Chen & Wei (2026) across recursions: $h_{\text{in}}^{(t+1)} = \mathcal{N}_2(R h_{\text{in}}^{(t)} + \mathcal{M}^L(\mathcal{N}_1(h_{\text{in}}^{(t)})))$.

Here $\mathcal{N}$, $\mathcal{N}_1$, and $\mathcal{N}_2$ are RMSNorms. Data, seed, batch, and learning-rate schedule match the main runs. LPT keeps Ouro’s normalization and final loss.

At every size and depth in Table 5, LPT with Ouro’s normalization reaches lower perplexity than the best LT, and the best LT reaches lower perplexity than our main LT. The alternatives are also more robust: every finished run trains with final loss at every size and depth, while Ouro’s normalization degrades or diverges. No single normalization is best for LT: update norm wins three settings and KEEL the other three. Choosing the best one lowers LT’s perplexity by 0.13–0.76 relative to our main LT. LPT still leads the best LT by 0.31–0.37 at S and by 0.06–0.43 at M. The smallest gap, 0.06

Table 5: **LPT outperforms LT with the best normalization we tried.** Validation perplexity ($\downarrow$) after 20B tokens. **Bold** marks the best per column and underline the best LT. Values above 2,000 are unigram-level. A dash marks a run that did not finish.

Model	Normalization	Loss	S			M		
			$R = 2$	$R = 3$	$R = 5$	$R = 2$	$R = 3$	$R = 5$
LT	Ouro (main)	Adaptive	16.17	15.98	16.00	13.28	13.13	13.34
	Ouro	Final	16.05	16.32	2,040	17.10	31.74	2,090
	Pre-norm only	Final	16.28	15.99	15.77	13.28	13.05	12.83
	Sandwich only	Final	16.23	15.83	15.67	—	—	—
	Input norm	Final	16.10	15.79	15.49	13.05	12.88	13.02
	Update norm	Final	<u>16.04</u>	<u>15.65</u>	15.35	13.01	<u>12.75</u>	12.77
	KEEL	Final	16.10	15.73	<u>15.24</u>	<u>12.96</u>	12.78	<u>12.76</u>
LPT	Ouro	Final	15.73	15.28	14.91	12.91	12.55	12.33

at M with two recursions, is within the seed variation we measure at S (Appendix B). At S, every alternative keeps improving through five recursions, unlike our main LT. At M, the best LT stops improving after three recursions.

Although this comparison favors LT, LPT still performs best. LT gets the best of six normalizations, chosen on the same validation perplexity we report, while LPT keeps Ouro’s. The same search for LPT would include Ouro’s normalization, so it could only keep or lower LPT’s perplexity. Since normalization is not the focus of our investigation, we keep Ouro’s for the main LT, as in the original paper.

D ADDITIONAL ANALYSIS DETAILS

Attention Representations For Figure 6, we use S-size models with three recursions and final loss. We evaluate each model on the same 4,096-token validation sequence. At each layer, we concatenate the attention heads into one query vector and one key vector per token, before rotary encoding. We pool these vectors across all tokens and recursions, subtract their common mean, and fit PCA by singular value decomposition, separately for each model and layer. We do not normalize the vectors or rescale their features. All recursions within a panel therefore share one basis. Axes are not aligned across models or layers.

We display the first two principal components. For readability, the panels show one token in 24, while PCA and the large centroid markers use all tokens. Lines connect centroids across recursions, separately for queries and keys. Axis limits are shared across layers within each model, with equal scaling on both axes.

Figure 9 adds LPT with full prediction caches. Figure 10 compares LPT across sizes and recursion counts, matching layers by relative depth. Figure 11 compares attention to shared memory across window sizes and training objectives. Each layer’s attention mass is averaged over queries and heads. We display six layers at matched relative depths, but compute the mean over all 12 layers for S and all 24 for M.

Gradient Feedback through Shared Memory We differentiate the mean cross-entropy of the final recursion at 32 sampled positions in each of 32 validation sequences of length 4,096. The sequence contents and loss positions are identical across conditions. Figure 7 uses the main-comparison checkpoints: LT trained with adaptive loss and LPT with final loss. STD provides a single-pass reference, measured through its native forward pass on the same data. Figure 12 adds LPT with full prediction caches and LPT trained with adaptive loss. Every probe differentiates the loss at the final recursion, regardless of the training objective. We measure gradients on Transformer block parameters, excluding embeddings, readout, recurrent normalization, and the halting gate. Each model retains its trained normalization.

For the backward computation, we give each recursion an independent parameter copy with the same values. This leaves the forward pass unchanged and isolates each recursion’s contribution to the shared-parameter gradient. Detaching shared KV at later reads isolates the first recursion’s

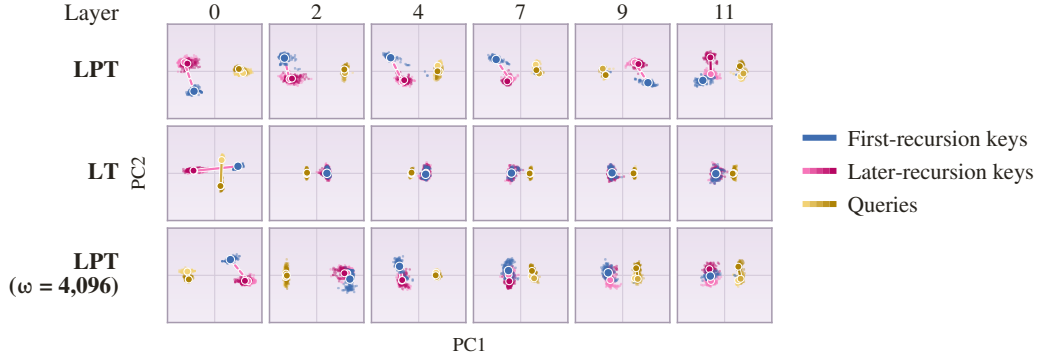


Figure 9: **Shared and local keys remain distinguishable with full prediction caches.** Expanded comparison for Figure 6, including $\omega = 4,096$.

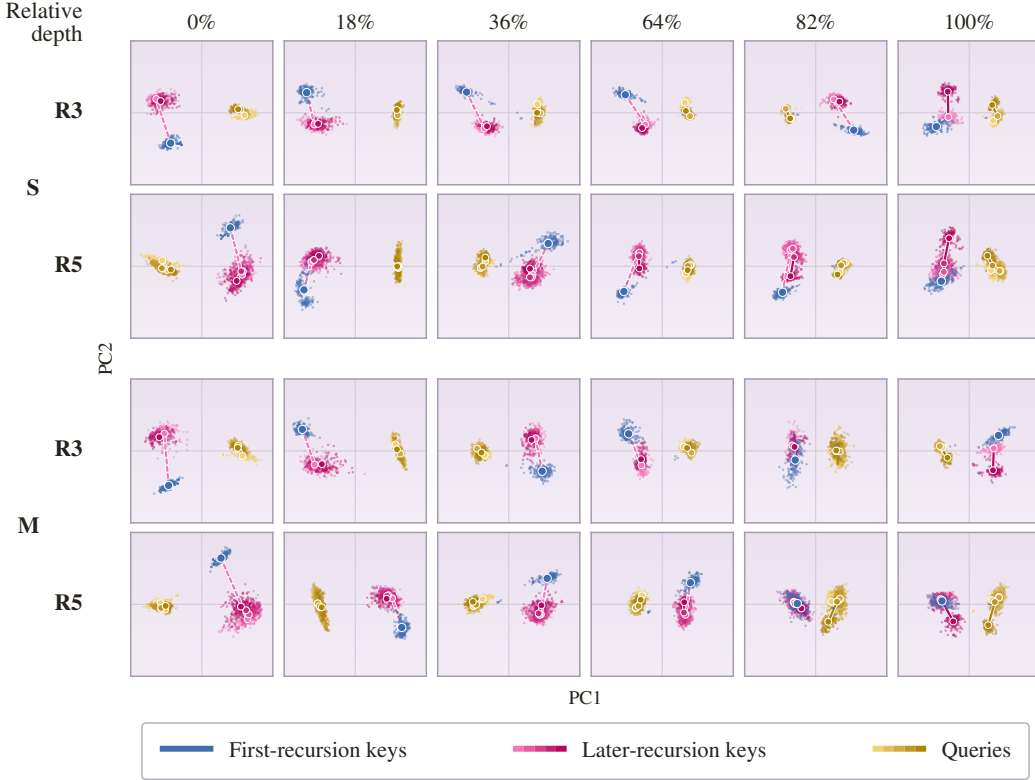


Figure 10: **Query/key geometry across LPT models.** S and M models at three and five recursions, with final loss and $\omega = 64$. Columns match relative layer depth.

hidden-state path contribution, g_{hidden} . Subtracting it from the ordinary first-recursion gradient $g^{(1)}$ gives the shared-cache contribution, g_{KV} . We verify that contributions from all parameter uses sum to the ordinary shared-parameter gradient.

The bars show arithmetic mean gradient norms across sequences. To split the first bar, we project each path onto its total gradient within each sequence:

$$p_a = \frac{\langle g_a, g^{(1)} \rangle}{\|g^{(1)}\|_2}, \quad a \in \{\text{KV}, \text{hidden}\}, \quad p_{\text{KV}} + p_{\text{hidden}} = \|g^{(1)}\|_2. \quad (5)$$

We then average these projected contributions. Their sum equals the mean first-recursion norm. The two path norms themselves are not additive. The projections are nonnegative in all plotted

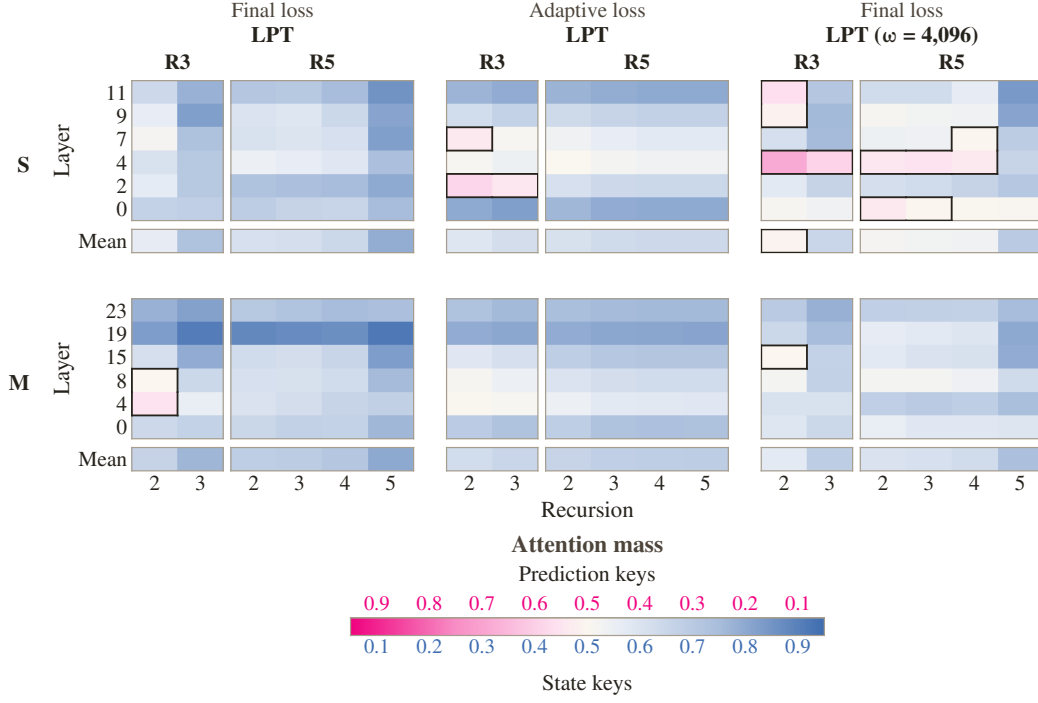


Figure 11: **Attention to shared memory across training objectives and window sizes.** Expanded comparison for Figure 5. Mean rows average all layers.

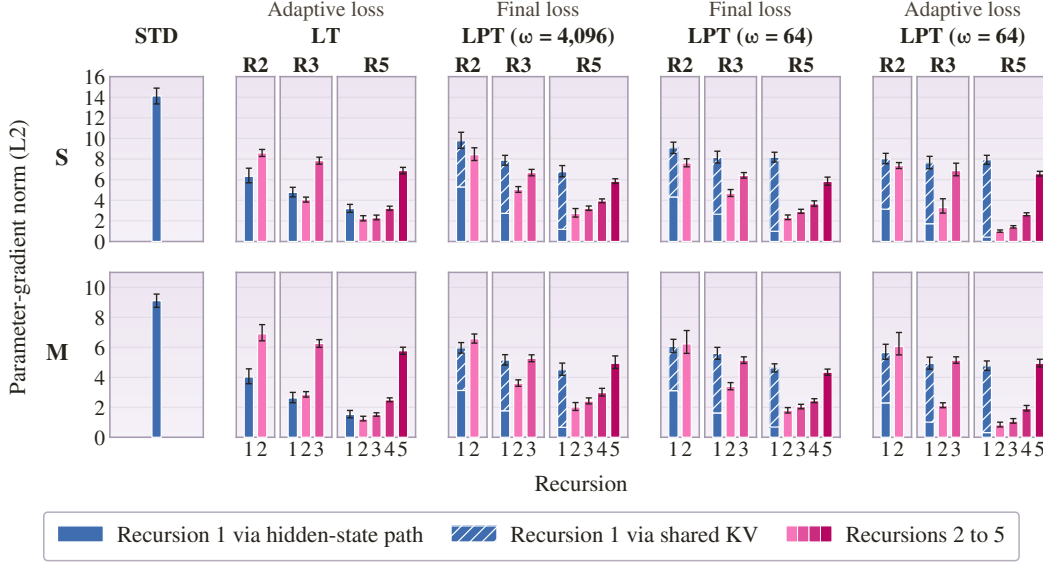


Figure 12: **Shared-cache feedback persists across training objectives and window sizes.** Expanded version of Figure 7, with the same sequences and final-loss gradient probe in every condition.

measurements, without clipping. Whiskers are 95% sequence-bootstrap intervals for the full bar, not for its segments or for variation across training seeds.

Figure 12 shows that the shared-cache contribution remains strong with a full prediction window and under adaptive loss. This is consistent across S and M. The compact main-paper comparison therefore does not depend on using final loss or a short prediction window for LPT.

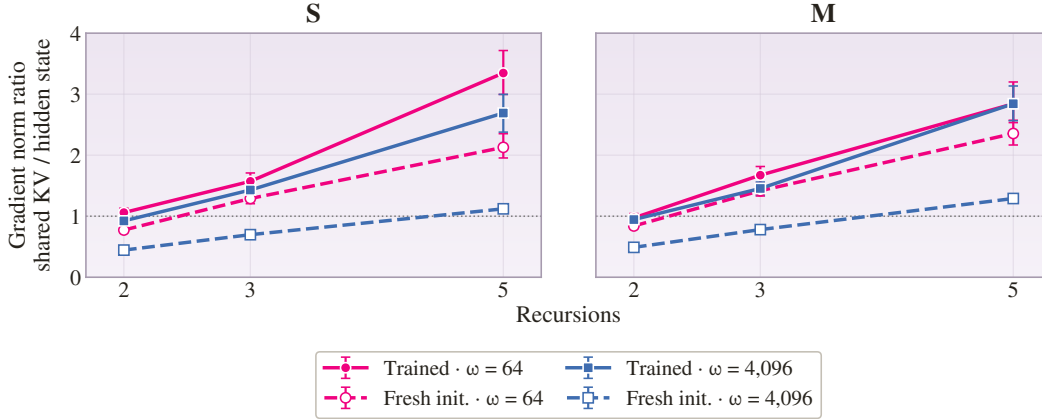


Figure 13: **Shared-KV feedback relative to the hidden-state path.** Gradient-norm ratios at fresh initialization and after training. Whiskers: 95% sequence-bootstrap intervals.

Gradient Paths at Initialization Figure 13 compares the norm ratio $\|g_{KV}\|_2 / \|g_{\text{hidden}}\|_2$ before and after training, using geometric means over the same 32 sequences. Fresh initialization is measured at two, three, and five recursions for both windows and sizes. It uses separate random weights, not saved initial checkpoints. The ratio increases with depth even at initialization, but is larger in the trained models. More recursions add more shared-cache reads, so this trend alone does not establish that the feedback helps learning. The detachment experiment tests that separately.

Shared-Cache Gradient Ablation We train S and M LPT models with $R \in \{2, 3, 5\}$, $\omega = 64$, and final loss, changing only whether the shared KV entries are detached at each read by a later recursion. The first recursion still receives gradients through the hidden-state path, and all shared parameters remain trainable. This removes direct gradient feedback through shared-cache reads, not all training of the first recursion.

Table 6 compares final-loss LT, detached LPT, and LPT after 20B training tokens. Detachment raises perplexity in every comparison. Detached S at five recursions finishes well above LPT, while M at three and five recursions remains near unigram-level loss. The experiment supports a role for shared-cache gradients in learning and stability, but does not distinguish shorter gradient paths from changes in gradient scale or direction.

Table 6: **Detaching shared-cache gradients hurts learning.** Validation perplexity ($\downarrow$) at S and M, all models trained with final loss; values above 1,000 are runs that failed to train. Bold marks the best per column.

Model	S			M		
	$R = 2$	$R = 3$	$R = 5$	$R = 2$	$R = 3$	$R = 5$
LT (final loss)	16.05	16.32	2,040	17.10	31.74	2,090
LPT (detached KV)	16.34	16.14	33.21	13.87	1,995	1,527
LPT	15.73	15.28	14.91	12.91	12.55	12.33

E EXPERIMENTS AT NINE RECURSIONS

Table 7 and Figure 14 extend the comparisons to nine recursions. We also list SPS (Monea et al., 2026) in the $R = 2$ column, since its memory and FLOPs match those of LPT with two recursions. From five to nine, LPT gains 0.7 accuracy points at S and 0.2 at M, but loses 1.7 at L. Perplexity improves at S and worsens at M and L. LT and PLT also worsen in perplexity at every size. Both hybrids continue to improve: H-LPT gains 0.4, 0.2, and 0.6 accuracy points at S, M, and L, respectively, and reaches the lowest perplexity of all models at every size. At nine recursions, LPT still outperforms LT at every size, and H-LPT outperforms LT2 in both perplexity and accuracy.

Higher recurrent depth can also complicate training. Ouro reports loss spikes and gradient oscillations at eight recursions and reduces its recurrent depth to four (Zhu et al., 2025). LT2 also uses four recursions (Deng et al., 2026).

Table 7: **Quality and efficiency through nine recursions.** One table per size. Metrics and formatting follow Table 1: in the STD row, $R = 1$ is the standard Transformer and $R > 1$ is DEEPSTD. SPS appears in the $R = 2$ column, whose memory and FLOPs it matches.

S	$R = 1$				$R = 2$				$R = 3$				$R = 5$				$R = 9$			
	Model	PPL ↓	Acc. ↑		PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓
STD		16.83	51.5		17.07	50.2	1.50	1.19	17.27	50.7	1.88	1.31	17.69	50.4	2.50	1.54	18.32	49.1	3.38	1.81
SPS					16.18	<u>52.6</u>	1.02	1.84												
LOOPIE					16.58	51.6	2.00	1.83	16.50	51.9	3.00	2.66	16.56	52.1	5.00	4.32	16.98	51.0	9.00	7.64
LT					16.17	51.5	2.00	1.83	15.98	52.1	3.00	2.66	16.00	52.4	5.00	4.32	16.13	52.2	9.00	7.64
PLT					15.84	52.7	1.02	1.84	15.55	52.9	1.03	2.67	15.49	53.5	1.06	4.34	15.51	53.2	<u>1.12</u>	7.68
LT2					<u>15.76</u>	51.9	<u>0.35</u>	<u>1.29</u>	15.47	52.5	<u>0.52</u>	<u>1.86</u>	15.16	53.7	<u>0.87</u>	<u>2.98</u>	14.96	54.4	1.56	<u>5.23</u>
LPT					15.73	51.9	1.02	1.84	15.28	54.0	1.03	2.67	14.91	54.9	1.06	4.34	<u>14.80</u>	55.6	<u>1.12</u>	7.68
H-LPT					15.73	52.4	0.18	<u>1.29</u>	<u>15.39</u>	<u>53.1</u>	0.19	<u>1.86</u>	<u>15.01</u>	<u>54.6</u>	0.21	<u>2.98</u>	14.74	<u>55.0</u>	0.25	<u>5.23</u>

M	$R = 1$				$R = 2$				$R = 3$				$R = 5$				$R = 9$			
	Model	PPL ↓	Acc. ↑		PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓
STD		13.79	56.1		14.24	55.5	1.44	1.13	14.47	55.4	1.78	1.24	14.88	54.4	2.34	1.43	15.60	53.4	3.09	1.63
SPS					12.99	58.9	1.02	1.93												
LOOPIE					13.69	56.9	2.00	1.92	13.75	55.8	3.00	2.84	13.74	56.6	5.00	4.69	15.80	52.9	9.00	8.37
LT					13.28	57.5	2.00	1.92	13.13	59.4	3.00	2.84	13.34	57.9	5.00	4.69	13.72	56.9	9.00	8.37
PLT					13.00	<u>59.1</u>	1.02	1.93	12.68	60.0	1.03	2.85	12.65	60.5	1.06	4.71	12.87	59.5	<u>1.12</u>	8.41
LT2					<u>12.81</u>	<u>59.1</u>	<u>0.35</u>	<u>1.43</u>	12.63	59.5	<u>0.52</u>	<u>2.10</u>	12.43	60.2	<u>0.87</u>	<u>3.45</u>	<u>12.31</u>	60.4	1.56	<u>6.14</u>
LPT					12.91	59.2	1.02	1.93	<u>12.55</u>	<u>60.2</u>	1.03	2.85	<u>12.33</u>	61.6	1.06	4.70	12.39	61.8	<u>1.12</u>	8.41
H-LPT					12.80	58.5	0.18	<u>1.43</u>	12.54	60.3	0.19	<u>2.10</u>	12.31	<u>61.4</u>	0.21	<u>3.45</u>	12.14	<u>61.6</u>	0.25	<u>6.14</u>

L	$R = 1$				$R = 2$				$R = 3$				$R = 5$				$R = 9$			
	Model	PPL ↓	Acc. ↑		PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓	PPL ↓	Acc. ↑	KV ↓	FLOPs ↓
STD		11.35	62.5		11.62	62.5	1.45	1.14	11.96	61.3	1.73	1.17	12.27	61.4	2.25	1.32	12.66	59.8	3.15	1.63
SPS					10.77	65.5	1.02	1.96												
LOOPIE					11.26	63.5	2.00	1.95	11.28	63.9	3.00	2.91	12.26	61.0	5.00	4.81	11.53	63.2	9.00	8.63
LT					11.20	64.2	2.00	1.95	11.11	63.8	3.00	2.91	11.39	63.4	5.00	4.81	12.52	60.2	9.00	8.63
PLT					10.95	64.8	1.02	1.96	10.75	66.8	1.03	2.92	10.59	66.4	1.06	4.83	10.78	66.1	<u>1.12</u>	8.66
LT2					<u>10.59</u>	66.5	<u>0.40</u>	<u>1.53</u>	10.47	65.8	<u>0.60</u>	<u>2.27</u>	<u>10.32</u>	67.3	<u>1.01</u>	<u>3.75</u>	<u>10.20</u>	<u>68.1</u>	1.81	<u>6.71</u>
LPT					10.75	<u>66.3</u>	1.02	1.96	<u>10.45</u>	67.3	1.03	2.92	<u>10.32</u>	68.2	1.06	4.83	10.87	66.5	<u>1.12</u>	8.66
H-LPT					10.58	65.6	0.21	<u>1.53</u>	10.41	<u>66.9</u>	0.22	<u>2.27</u>	10.23	<u>68.1</u>	0.24	<u>3.75</u>	10.11	68.7	0.28	6.72

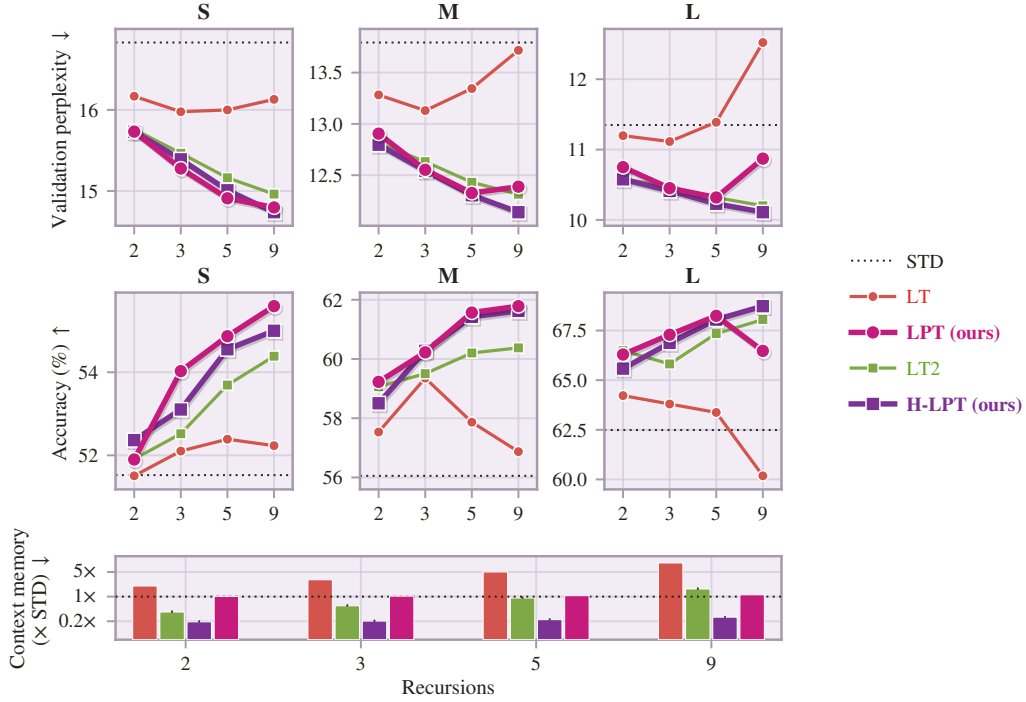


Figure 14: **Quality and memory through nine recursions.** Validation perplexity, mean five-task accuracy, and context memory relative to STD.

Intermediate Readouts We apply the model’s readout normalization and language-modeling head to the output of each recursion. The measurements use S-size LPT checkpoints trained for 20B tokens, averaged over 16 validation sequences of 4,096 tokens. With nine recursions and final loss, intermediate-readout perplexity ranges from 87 to 1,012, falling to 15.6 at the last recursion. With adaptive loss, it is already 2.79 by the fourth recursion and remains near that value. Under final loss, the head is trained only on the last recursion: high intermediate perplexity measures poor prediction through this head, not the absence of predictive information in the hidden state. These probe-set losses are separate from the full validation-set perplexity in Table 7.

Complete Quality–Memory Sweep Figure 15 includes every family, size, and recursion count in Table 7 except SPS and PLT. Figure 16 compares nine-recursion models with STD across sizes.

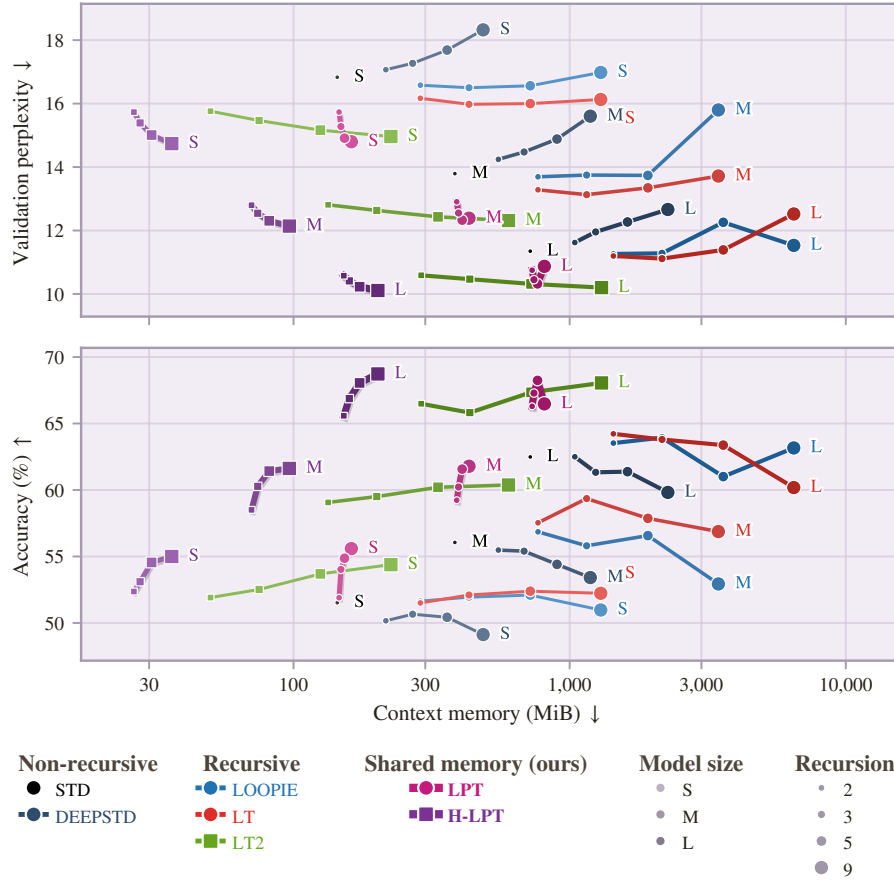


Figure 15: **Quality versus context memory across model sizes.** Curves vary recursion count. Marker size denotes recursions, and shade denotes model size. S/M train for 20B tokens and L for 50B.

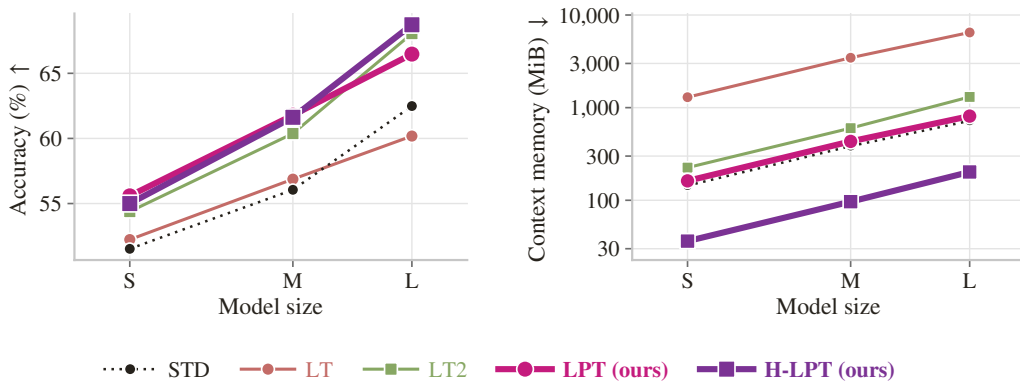


Figure 16: **Quality and memory across model sizes.** Mean five-task accuracy and context memory for STD and looped models at nine recursions.

F DECODING SPEED AND MEMORY ON A GPU

We test whether the analytic memory savings translate into larger batches and higher decoding throughput on a GPU. A smaller cache lets more sequences share a GPU, which can raise throughput (Brandon et al., 2024; Sun et al., 2024). As O’Neill & Reid (2026) do for a looped model, we measure the largest batch that fits at a fixed context and the throughput at that batch.

Setup We decode on one NVIDIA H100 with 80 GB of memory, in bf16, at a 4,096-token context: each step generates one token per sequence from a cache already filled to 4,068 tokens. All models share one decoding implementation in PyTorch: a preallocated cache per attention layer, FlexAttention (Dong et al., 2025) for attention, and CUDA graphs that replay a whole decoding step, all recursions and layers included. A later recursion of LPT or H-LPT attends to its shared and local caches in one softmax, which we compute by merging the two FlexAttention outputs by their log-sum-exp. Gated DeltaNet layers update their recurrent state one token at a time, with the kernels of flash-linear-attention (Yang & Zhang, 2024). STD runs the same code with one recursion, and LT runs all its recursions, without early exit. Weights are random, since speed and memory do not depend on them. On the same GPU, decoding reproduces the full forward pass to bf16 precision. The maximum batch is the largest batch whose decoding steps run without running out of memory, and we measure throughput at this batch, or at 95% of it when the CUDA graph does not fit. These measurements come from one research implementation, not an optimized serving engine, which would change the absolute speeds. Every model runs the same code, so the comparison holds within this implementation.

Capacity Table 8 shows that the analytic savings carry over to the GPU. LPT’s cache per sequence is at most 6.25% larger than STD’s, so it fits 93–98% of STD’s batch. LT’s cache grows with depth, so LPT fits 2.0, 2.9, and 4.7–4.8 times as many sequences as LT at two, three, and five recursions. H-LPT holds 20–27% of STD’s cache and fits 3.6–5.0 times as many sequences as STD, and 1.8–3.8 times as many as LT2. The hybrids’ measured caches are 3–17% larger than the analytic values in Table 1, because the kernel keeps the recurrent state in fp32 and we also count the short-convolution windows.

Speed The larger batches raise LPT’s peak throughput above LT’s by 7–91%, most at L with five recursions, where LT fits only 21 sequences. At equal batch, LPT is 3–8% slower than LT, since each later recursion still reads the whole shared cache, and its own window too. As Brandon et al. (2024) note for sharing across layers, a shared cache saves memory, not the time to read it. Both looped models pay for depth in time: at batch 1, a decoding step of LT takes about 2.3, 3.4, and 5.7 times as long as STD’s at two, three, and five recursions, and LPT’s takes 8–12% longer than LT’s. The hybrids read a full-context cache in only one layer of five, so they decode faster. At its maximum batch, H-LPT generates 2.1–2.2 times as many tokens per second as STD with two recursions, 1.4–1.5 times with three, and 0.8–0.9 times with five, and it is 2–3% slower than LT2 at equal batch.

Throughput and Quality Figure 17 plots quality against throughput. At every size and recursion count, LPT has higher throughput, lower perplexity, and higher accuracy than LT, and H-LPT has higher throughput and lower perplexity than LT2. Against STD, H-LPT lowers perplexity by 0.94–1.44 with three recursions while decoding 1.4–1.5 times as many tokens per second, and by 1.12–1.82 with five, at 81–88% of STD’s throughput.

Table 8: **Decoding capacity and speed on one H100.** One table per size. bf16, 4,096-token context. Batch: the largest batch that decodes without running out of the 80 GB. Tok/s: generated tokens per second at that batch. Lat.: milliseconds per decoding step at batch 1. Our rows are shaded.

S	R = 1			R = 2			R = 3			R = 5		
Model	Batch	Tok/s	Lat.	Batch	Tok/s	Lat.	Batch	Tok/s	Lat.	Batch	Tok/s	Lat.
STD	552	18,570	2.1									
LT				275	8,414	4.8	184	5,172	7.1	110	2,688	11.8
LPT				540	8,975	5.2	533	5,894	7.9	514	3,544	13.3
LT2				1,513	39,006	2.6	1,007	25,470	3.9	603	14,348	6.4
H-LPT				2,746	40,066	2.7	2,531	26,692	4.0	2,190	15,900	6.7

M	R = 1			R = 2			R = 3			R = 5		
Model	Batch	Tok/s	Lat.	Batch	Tok/s	Lat.	Batch	Tok/s	Lat.	Batch	Tok/s	Lat.
STD	206	6,478	4.2									
LT				103	2,703	9.7	68	1,527	14.5	41	763	24.1
LPT				202	3,074	10.5	199	2,000	16.0	192	1,201	27.0
LT2				563	13,804	5.3	377	8,511	8.0	224	4,502	13.2
H-LPT				1,029	14,549	5.5	949	9,622	8.2	821	5,706	13.6

L	R = 1			R = 2			R = 3			R = 5		
Model	Batch	Tok/s	Lat.	Batch	Tok/s	Lat.	Batch	Tok/s	Lat.	Batch	Tok/s	Lat.
STD	108	3,162	6.7									
LT				54	1,203	15.3	36	691	22.8	21	303	37.9
LPT				106	1,480	16.4	104	976	25.0	101	578	42.3
LT2				258	5,986	8.8	172	3,585	13.1	103	1,776	21.7
H-LPT				475	6,612	9.0	444	4,333	13.6	390	2,553	22.6

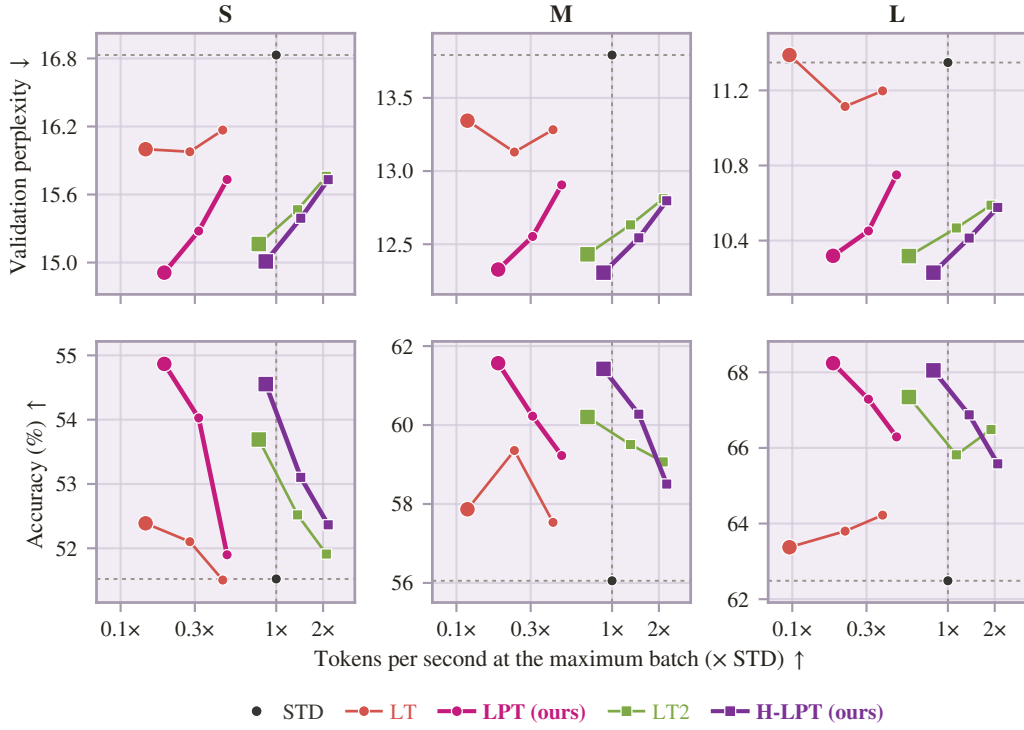


Figure 17: **H-LPT is better than STD at similar or higher throughput.** Validation perplexity (top) and mean five-task accuracy (bottom) against generated tokens per second at each model’s maximum batch, relative to same-size STD, on one H100 at a 4,096-token context. Curves follow two, three, and five recursions, with larger markers at five. Dashed lines mark STD.